EOAT Command Center Codex Phase Prompt Book

Purpose

This document contains a full Codex-ready prompt for each development phase in the EOAT Command Center improvement roadmap.

The goal is to give Codex one controlled phase at a time instead of asking it to build the whole universe, discover consciousness, and refactor the audit page before lunch. Each prompt is designed to be copied into Codex as a standalone instruction set.

The prompts are intentionally detailed. They tell Codex what to inspect, what to preserve, what to implement, what to test, what not to touch, and what to report back.

Critical Global Rules for Every Codex Phase

Use these rules in every Codex session, even if the phase prompt repeats them.

Absolute Safety Rules

1. Do not commit or generate real company data into the repository.
2. Do not commit real workbooks, real reports, real photos, local config files, private network paths, capacity files, downtime data, scrap data, cycle-time data, customer names, mold numbers, part numbers, maintenance histories, or internal operational details.
3. Use only sanitized templates, synthetic demo data, and generic examples inside the repo.
4. Keep real project data outside the repo.
5. Preserve local-first behavior. Do not add cloud APIs or internet dependencies.
6. Do not break existing CLI tools.
7. Do not bury business logic inside UI page files when it belongs in core modules.
8. Every workbook-modifying operation must create a backup unless there is a documented reason it is safe not to.
9. Every destructive or cleanup action must require confirmation.
10. Unknown / Not Checked is not the same as N/A.
11. N/A means the field does not physically apply.
12. Blank means the user missed it or the app failed to save properly.
13. Follow-up means the field applies but needs later review.
14. Verified means it was actually observed or intentionally confirmed.
15. Cached data must be clearly labeled as fresh, stale, missing, or unknown.
16. Reports should prefer real activity logs, audit saves, validation findings, notes, tags, open items, and completed tasks over generic filler text.
17. Add or update tests for every behavior change.
18. Update documentation when user-facing behavior changes.
19. Preserve demo project behavior.

20. Keep the app useful even when optional data sources are missing.

Robot Info Scope Rule

Do not implement a full Robot Info entity system.

Robot_Info.xlsx must remain a small workbook only for these robot-side pneumatic circuit details:

- Robot Vacuum Circuits.
- Robot Pressure Circuits.
- Robot Interchangeable Circuits.

It may also contain basic tracking fields such as:

- Plant/Area.
- Machine Number.
- Last Audit ID.
- Last Updated.
- Notes.

Do not duplicate Master Press List robot details into Robot_Info.xlsx. The Master Press List remains the source for deeper robot reference details.

Required Final Codex Response Format

At the end of each phase, Codex should report:

```
Summary
- What was implemented.

Files Changed
- File path: short explanation.

Tests Run
- Command and result.

Manual Smoke Checks
- What was checked manually, if anything.

Safety Checks
- Repo safety audit status.
- Whether any generated/private/local files were avoided.

Known Risks / Follow-Ups
```

```
- Anything not completed.
- Any behavior that needs manual review.
```

Recommended Development Pattern

For every phase, Codex should work in this order:

1. Inspect relevant files.
2. Summarize current architecture.
3. Propose the smallest safe implementation plan.
4. Implement in logical chunks.
5. Add/update tests.
6. Run relevant tests.
7. Run safety checks where practical.
8. Summarize changes and risks.

Phase 0 Prompt — Current-State Audit and Safety Baseline

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 0 — Current-State Audit and Safety Baseline from the EOAT
Command Center roadmap.

This phase is inspection, documentation, and safety baseline only. Do not
refactor major app architecture yet. Do not add large new features yet. The
purpose is to document the current state before bigger changes begin.

Critical constraints:
- Do not commit or generate real company data into the repository.
- Do not commit real workbooks, real reports, real photos, local config files,
private network paths, capacity files, downtime data, scrap data, cycle-time
data, customer names, mold numbers, part numbers, maintenance histories, or
internal operational details.
- Use only sanitized templates, synthetic demo data, and generic examples.
- Preserve current app behavior.
- Preserve existing CLI tools.
- Do not implement the full Robot Info entity system. Robot_Info.xlsx must
remain a small workbook only for robot-side pneumatic circuit counts.
```

```
Before editing code:
1. Inspect the repository structure.
2. Identify the current GUI entry point.
3. Identify standalone CLI/tool entry points.
4. Identify current scheduled report scripts.
5. Identify current workbook validation flow.
6. Identify current audit save/load flow.
7. Identify current notes/tags/annotation modules.
8. Identify current test layout.
9. Identify existing docs related to setup, usage, safety, and scheduled
reports.

Required work:

1. Create a current architecture inventory document.
   Suggested path:
   docs/current_state_architecture_inventory.md

   It should include:
   - app/ pages and purpose.
   - app/widgets and purpose.
   - core/ modules and purpose.
   - tools/ scripts and purpose.
   - scripts/ scheduled report and utility scripts.
   - data_templates and templates.
   - examples/demo_project role.
   - tests structure.
   - workbook-related modules.
   - annotation-related modules.
   - scheduled-report-related modules.

2. Create an entry point and workflow map document.
   Suggested path:
   docs/current_state_entry_points_and_workflows.md

   It should include:
   - GUI startup path.
   - dashboard launch command.
   - app initialization sequence.
   - audit save workflow.
   - audit load workflow.
   - compatibility workflow.
   - small Robot_Info.xlsx circuit workflow.
   - scheduled daily/weekly report workflow.
   - workbook validation workflow.
   - notes/tags/annotation workflow.
   - demo project behavior.
```

```
3. Create or update a manual smoke-test checklist.
   Suggested path:
   docs/manual_smoke_test_checklist.md

   Include checks for:
   - app launch.
   - Home page load.
   - project root display.
   - demo mode warning.
   - audit page load.
   - load existing audit.
   - save audit.
   - generate new audit ID.
   - machine lookup.
   - compatibility update.
   - Robot_Info.xlsx small circuit update.
   - notes page load.
   - tags page load.
   - scheduled reports page load.
   - manual daily summary dry run if available.
   - manual weekly summary dry run if available.
   - workbook validation.
   - dashboard quick refresh.
   - dashboard deep refresh.

4. Run the current test suite or the most practical subset.
   Preferred:
   python -m pytest

   If the full suite is too slow or blocked, run a focused subset and document
why.

5. Run the repo safety audit if available.
   Preferred:
   python scripts/repo_safety_audit.py

6. Document baseline test and safety results.
   Suggested path:
   docs/current_state_baseline_test_results.md

   Include:
   - commands run.
   - pass/fail summary.
   - known failures.
   - warnings.
   - skipped checks.
   - any environment limitations.
```

```
7. Do not change app behavior unless needed to fix a documentation/test command
typo.

Acceptance criteria:
- Current architecture is documented.
- Current entry points/workflows are documented.
- Manual smoke-test checklist exists or is updated.
- Existing tests are run or a reason is documented.
- Repo safety audit is run or a reason is documented.
- No private data is added.
- No major behavior changes are made.

Final response requirements:
- List files created/changed.
- List commands run and results.
- List known risks or suspicious areas discovered.
- Confirm whether repo safety audit passed, failed, or could not be run.
```

Phase 1 Prompt — App Architecture Foundation

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 1 — App Architecture Foundation from the EOAT Command Center
roadmap.

Included features:
- #35 App-level event bus.
- #36 Page registry.
- #37 Page lifecycle hooks.

Goal:
Create a cleaner internal app structure so future features can plug in without
every page manually knowing about every other page.

Critical constraints:
- Preserve existing visible app behavior.
- Do not redesign the UI yet.
- Do not break current navigation.
- Do not break CLI tools.
- Do not add cloud or internet dependencies.
- Do not commit real company data.
- Do not implement the full Robot Info entity system.
```

```
Before editing code:
1. Inspect app/dashboard_ui.py.
2. Inspect app/navigation.py.
3. Inspect the existing app/pages modules.
4. Inspect app/page_tasks.py and task runner behavior.
5. Inspect settings/project-root handling.
6. Inspect tests/ui or dashboard-related tests.
7. Summarize how pages are currently registered, created, shown, and refreshed.

Required work:

1. Add a central page registry.
   Suggested module:
   app/page_registry.py

   Define a PageSpec dataclass or equivalent with fields like:
   - key.
   - label.
   - section.
   - factory/import path or callable.
   - requires_config.
   - refresh_on_show.
   - listens_to.
   - description.

   The registry should define all existing pages currently in navigation.

2. Update navigation to build from the page registry.
   Existing navigation labels and sections should remain the same unless there
is an obvious typo.

3. Replace the hardcoded page factory / if-elif import chain in dashboard_ui
with a registry-driven loader.
   Requirements:
   - Lazy page loading must remain.
   - Placeholder loading labels can remain.
   - Missing/broken pages should fail clearly.
   - Existing pages should still receive config when they currently expect
config.

4. Add page lifecycle hook support.
   DashboardWindow should safely call optional methods if a page implements
them:
   - on_show()
   - on_hide()
   - on_project_root_changed(config)
   - on_event(event)
```

```
   - can_close()

   Do not require all pages to implement these methods.

5. Add an app event bus.
   Suggested module:
   app/event_bus.py

   Suggested objects:
   - AppEvent dataclass.
   - EventBus class.
   - global get_event_bus() function if consistent with existing style.

   Suggested event names:
   - AuditSaved
   - AuditLoaded
   - AnnotationChanged
   - TagChanged
   - TagColorSynced
   - RobotInfoUpdated
   - CompatibilityRegenerated
   - WorkbookValidated
   - ReportGenerated
   - DashboardCacheInvalidated
   - ProjectRootChanged
   - SettingsChanged

6. Wire event dispatch minimally.
   At minimum, emit events where existing workflows already clearly finish:
   - project root changed.
   - settings saved.
   - audit save success if easy and safe.
   - workbook validation success if easy and safe.

   Do not over-wire every page in this phase.

7. Update key pages to tolerate lifecycle hooks.
   At minimum:
   - Home page can refresh on show or on project root changed.
   - Notes/Tags can refresh on annotation/tag events if straightforward.
   - Audit Progress can refresh on AuditSaved if straightforward.

8. Add tests.
   Tests should cover:
   - page registry contains expected page keys.
   - navigation can be built from registry.
   - dashboard can instantiate at least Home and Audit page in smoke/offscreen
mode if existing test pattern supports it.
```

```
   - event bus dispatches to subscribers.
   - lifecycle hook calls do not crash when methods are absent.

9. Update documentation if needed.
   Add a small developer note:
   docs/page_registry_and_event_bus.md

   Include:
   - how to add a new page.
   - how to listen for events.
   - how page lifecycle hooks work.

Acceptance criteria:
- Dashboard navigation still works.
- Existing pages still open.
- Lazy loading still works.
- Adding a page requires editing the registry, not dashboard_ui factory logic.
- Event bus exists and is tested.
- Lifecycle hooks are optional and safe.
- No business logic is moved into dashboard shell.
- Tests pass or failures are documented.
- No private data added.

Run tests:
- Run focused tests for dashboard/navigation/event bus.
- Run broader pytest if practical.
- Run repo safety audit if practical.

Final response requirements:
- List files changed.
- Explain the new page registry.
- Explain event bus behavior.
- Explain lifecycle hooks.
- List tests run and results.
- List risks/follow-ups.
```

Phase 2 Prompt — Audit Workflow Stabilization

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 2 — Audit Workflow Stabilization from the EOAT Command Center
roadmap.
```

```
Included features:
- #5 Dirty-form protection.
- #6 Split AuditPage into smaller pieces.
- #7 Configurable defaults.
- #20 Field history and audit revision history, initial foundation.
- #29 Compatibility impact preview.
- #38 Operational Settings page, initial version.

Goal:
Make the audit workflow safer, cleaner, and more maintainable before adding
Audit Coach and richer intelligence.

Critical constraints:
- Preserve current audit save/load behavior.
- Preserve generated audit ID behavior.
- Preserve empty-only mode.
- Preserve field visibility rules.
- Preserve compatibility update behavior.
- Preserve annotation field indicators.
- Preserve small Robot_Info.xlsx circuit-count behavior only.
- Do not implement a full Robot Info entity system.
- Do not commit real company data.

Before editing code:
1. Inspect app/pages/audit.py.
2. Inspect core/audit_entries.py.
3. Inspect core/audit_field_rules.py.
4. Inspect core/audit_compatibility.py.
5. Inspect core/robot_info.py only to preserve the small circuit workbook
behavior.
6. Inspect settings/config modules.
7. Inspect tests for audit entries, audit workflow, compatibility, validation,
and UI audit workflow.
8. Summarize the current audit page responsibilities and identify the safest
split points.

Required work:

1. Split the audit page into smaller modules.
   Suggested structure:

   app/pages/audit/
   - __init__.py
   - page.py
   - audit_form_view.py
   - audit_form_controller.py
   - audit_form_model.py
```

```
   - audit_visibility_controller.py
   - audit_defaults_controller.py
   - machine_lookup_controller.py
   - compatibility_panel.py
   - interview_panel.py
   - annotation_field_adornments.py
   - audit_save_workflow.py
   - audit_history_panel.py

   Do not attempt a giant perfect rewrite. Move responsibilities in safe chunks.
   If the split is too large for one pass, prioritize:
   - compatibility panel extraction.
   - audit defaults controller.
   - audit save workflow.
   - dirty-form/draft support.

2. Move business/domain logic into core/audit/ where practical.
   Suggested modules:

   core/audit/
   - __init__.py
   - defaults.py
   - drafts.py
   - history.py
   - compatibility_preview.py
   - completion.py, placeholder only if Phase 3 will use it.

   Avoid putting workbook/domain rules directly into UI files.

3. Add dirty-form detection.
   Behavior:
   - Track baseline form values after new form reset, existing audit load,
duplicate creation, and successful save.
   - Detect unsaved changes when current values differ from baseline.
   - Warn before loading another audit if unsaved changes exist.
   - Warn before clearing form if unsaved changes exist.
   - Warn before closing/navigating away if page lifecycle supports can_close().
   - Options should include Save, Save Draft, Discard, Cancel where practical.

4. Add local draft recovery.
   Suggested storage:
   00_Project_Admin/cache/audit_drafts/

   Draft file should include:
   - version.
   - saved_at.
   - project_root.
   - audit_id.
```

```
   - mode.
   - form_values.
   - baseline_values.

   Behavior:
   - Save draft manually through dirty-form warning.
   - Auto-save periodically if easy and safe.
   - Restore draft on app start or audit page open if present.
   - Allow discard draft.
   - Draft files must be ignored by repo safety rules.

5. Move hardcoded audit defaults into configuration/reference data.
   Defaults to move:
   - default auditor.
   - default Plant/Area.
   - default Cleanroom/Non-Cleanroom.
   - default Status.
   - default Priority.
   - Follow-Up Needed default.
   - Quick Disconnects Present default.
   - Pneumatic Quick Disconnect Type default.
   - Vacuum Generator Type default.
   - EOAT Interchangeable Circuits default.
   - Robot Interchangeable Circuits default.
   - documentation/photo default values.
   - reliability unknown defaults.
   - connection type to changeover difficulty defaults.

   Preserve current default behavior unless the existing desired requirement
says otherwise.

6. Add Operational Settings page, initial version.
   If Settings already exists, expand it.

   Initial sections:
   - Project root.
   - Audit Defaults.
   - Connection Defaults.
   - Scheduled Reports summary/read-only if not editable yet.
   - Safety/Backups summary/read-only if not editable yet.

   The Settings page should save to ignored local config, not committed files.

7. Add compatibility impact preview.
   Before saving an edited physical audit that has compatibility rows tied to
it, show a preview:
   - source audit ID.
   - number of compatible rows.
```

```
   - fields likely to propagate.
   - whether compatibility regeneration will run.
   - whether generated press views may refresh.

   Provide choices:
   - Save and update compatibility.
   - Save audit only if existing architecture supports it safely.
   - Cancel.

   If Save audit only is unsafe or not supported, do not offer it. Explain that
compatibility will update.

8. Add initial audit history logging.
   Suggested storage:
   00_Project_Admin/history/audit_history.jsonl

   Record:
   - timestamp.
   - audit_id.
   - event type.
   - changed fields.
   - old values.
   - new values.
   - source.
   - files modified if available.

   Keep this local/project-output data, not repo data.

9. Add tests.
   Tests should cover:
   - dirty detection true/false.
   - draft save/load/discard.
   - defaults loaded from config/reference data.
   - compatibility preview detects compatible rows.
   - audit history records changed fields.
   - existing audit save/load still works.
   - existing compatibility workflow still works.
   - small Robot_Info.xlsx circuit workflow still works.

10. Update documentation.
    Add or update:
    - docs/audit_workflow_safety.md
    - docs/settings_defaults.md

Acceptance criteria:
- Existing audit workflows still work.
- User cannot easily lose unsaved audit edits.
- Draft recovery works with ignored local files.
```

```
- Defaults are not scattered through UI code.
- Compatibility impact preview appears before risky edit saves.
- Audit history records meaningful changes.
- Robot_Info.xlsx remains a small circuit workbook only.
- Tests pass or failures are documented.
- No private data added.

Run tests:
- Run audit-related tests.
- Run UI audit workflow tests if available.
- Run settings/config tests.
- Run repo safety audit if practical.

Final response requirements:
- List files changed.
- Explain what was split out of the audit page.
- Explain dirty-form and draft behavior.
- Explain defaults/config changes.
- Explain compatibility preview.
- Explain audit history storage.
- List tests run and results.
- List risks/follow-ups.
```

Phase 3 Prompt — Audit Coach and Guided Completion

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 3 — Audit Coach and Guided Completion from the EOAT Command
Center roadmap.

Included features:
- #1 Audit Coach Mode.
- #4 Finish This Audit button.
- #10 “Why is this field hidden?” explanations.

Goal:
Turn the audit page from a large dynamic form into a guided workflow that tells
the user what still matters, what is missing, and why.

Critical constraints:
```

```
- Use existing field applicability rules.
- Do not invent separate truth rules in the UI.
- Do not treat Unknown / Not Checked as the same as verified complete.
- Do not treat N/A as missing when the field physically does not apply.
- Preserve existing save/load behavior.
- Do not implement full Robot Info entity system.
- Do not commit real company data.

Before editing code:
1. Inspect current audit page modules after Phase 2.
2. Inspect core/audit_field_rules.py.
3. Inspect core/audit_entries.py validation/default behavior.
4. Inspect any audit completion/draft/history modules from Phase 2.
5. Inspect annotation/tag field indicator code.
6. Inspect tests for audit field visibility and validation.
7. Summarize how field applicability, hidden fields, and warnings currently
work.

Required work:

1. Add core Audit Coach model.
   Suggested module:
   core/audit/coach.py

   Suggested dataclasses:

   - AuditCoachSectionStatus:
     - section_name.
     - applicable_count.
     - completed_count.
     - missing_count.
     - unknown_count.
     - follow_up_count.
     - warning_count.
     - fields.

   - AuditCoachFinding:
     - severity.
     - category.
     - field.
     - message.
     - reason.
     - recommended_action.
     - action_type.
     - fix_available.

   - AuditCoachSummary:
     - audit_id.
```

```
     - machine.
     - eoat_type.
     - mode.
     - applicable_field_count.
     - completed_field_count.
     - missing_important_count.
     - section_statuses.
     - findings.
     - next_action.

2. Add audit completion calculation.
   It should classify fields as:
   - verified complete.
   - missing.
   - Unknown / Not Checked.
   - N/A because non-applicable.
   - needs follow-up.
   - conflict/stale.

   Completion must use applicability rules, not raw blank-cell counting.

3. Add section completion calculation.
   Use existing audit section/group definitions from the audit UI or move
section definitions into a shared module if needed.

   For each section calculate:
   - applicable fields.
   - completed fields.
   - missing fields.
   - unknown fields.
   - fields with warnings.

4. Add Next Best Field logic.
   Priority order:
   1. Required identity fields.
   2. EOAT type and tooling fields.
   3. Fields that control visibility of other fields.
   4. Major engineering fields.
   5. Sensor/pneumatic/electrical details.
   6. Documentation/photo evidence fields.
   7. Notes and optional fields.

5. Add Audit Coach UI panel.
   Suggested placement:
   - Right side of Audit Entry tab.
   - Collapsible side panel if space is tight.

   Display:
```

```
   - audit ID.
   - machine.
   - EOAT type.
   - mode.
   - completion count.
   - section completion summary.
   - top warnings.
   - hidden field reasons.
   - next best action.

6. Add Finish This Audit workflow.
   Add button:
   Finish This Audit

   Behavior:
   - Enter guided completion mode.
   - Jump to the next missing applicable field.
   - Explain why the field matters.
   - Provide actions:
     - Open Field.
     - Mark Unknown / Not Checked.
     - Create Follow-Up.
     - Tag Needs Review.
     - Skip.
     - Next.
   - Update completion score live.

7. Add hidden field explanation viewer.
   For hidden fields, show reason strings from core field rules.

   Example:
   - Cup Type/Material hidden because vacuum tooling fields do not apply to
Mechanical / Gripper EOAT.
   - Electrical Quick Disconnect Type hidden because Electrical/Wiring Present?
is No.

8. Add false-completeness protections.
   The coach should distinguish:
   - complete and verified.
   - complete with follow-up.
   - incomplete.
   - blocked.

   Unknown / Not Checked should not be counted as verified.

9. Add tests.
   Tests should cover:
   - AuditCoachSummary generation for Vacuum EOAT.
```

```
   - AuditCoachSummary generation for Mechanical / Gripper EOAT.
   - AuditCoachSummary generation for Hybrid EOAT.
   - hidden field reason correctness.
   - next best field order.
   - Finish This Audit missing-field list.
   - Unknown / Not Checked classification.
   - N/A non-applicable classification.
   - stale hidden value finding.

10. Update docs.
    Suggested path:
    docs/audit_coach_and_guided_completion.md

Acceptance criteria:
- Audit Coach appears on the audit page.
- Coach updates when form values change.
- Hidden fields are explainable.
- Completion scoring respects applicability rules.
- Finish This Audit guides through missing applicable fields only.
- Follow-up and unknown states are truthful.
- Existing audit workflows still work.
- Tests pass or failures are documented.
- No private data added.

Run tests:
- Run audit coach/core tests.
- Run audit UI smoke tests if available.
- Run validation tests because coach depends on rules.
- Run repo safety audit if practical.

Final response requirements:
- List files changed.
- Explain Audit Coach model and UI.
- Explain Finish This Audit workflow.
- Explain hidden field reason behavior.
- List tests run and results.
- List risks/follow-ups.
```

Phase 4 Prompt — Annotations, Notes, Tags, and Open Items

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 4 — Annotations, Notes, Tags, and Open Items from the EOAT
Command Center roadmap.

Included features:
- #2 Apply/reject annotation suggestions.
- #3 Unified Open Items / Action Board.

Goal:
Make annotations, notes, tags, validation warnings, and follow-ups work together
as one action-management system.

Critical constraints:
- Preserve existing annotation database behavior.
- Preserve existing notes/tags pages unless intentionally improved.
- Do not break workbook tag color sync.
- Do not commit annotation databases or real project data.
- Do not implement full Robot Info entity system.

Before editing code:
1. Inspect core/annotations modules.
2. Inspect notes page.
3. Inspect tags page.
4. Inspect field tag button/dialog widgets.
5. Inspect existing annotation suggestion code.
6. Inspect activity logging and action item modules.
7. Inspect validation output if structured findings exist from Phase 5 or
current validation warnings if Phase 5 has not happened yet.
8. Summarize how notes, tags, targets, assignments, and suggestions currently
work.

Required work:

1. Improve annotation suggestions UI.
   Replace text-only suggestion output with a structured table.

   Suggested columns:
   - Apply checkbox.
   - Tag.
```

```
   - Target field.
   - Reason.
   - Severity or confidence.
   - Suggested comment.
   - Existing note/tag status.

   Buttons:
   - Apply Selected.
   - Apply All High Confidence.
   - Ignore Selected.
   - Create Follow-Up Actions.
   - Open Target Field.

2. Add suggestion IDs or hashes.
   Suggestions need stable identity so ignored suggestions can be tracked.

   Suggested fingerprint inputs:
   - audit ID.
   - target field.
   - tag name.
   - reason.
   - relevant current field value.

3. Add ignored suggestion tracking.
   Suggested storage:
   - annotation database table, if appropriate.
   - or local project JSONL if simpler.

   Store:
   - suggestion ID/hash.
   - audit ID.
   - field.
   - tag.
   - reason.
   - ignored timestamp.
   - data fingerprint.

   Behavior:
   - Do not repeatedly show ignored suggestion unless underlying data changes.
   - Provide a way to show ignored suggestions if practical.

4. Add unified Open Items model.
   Suggested module:
   core/open_items.py

   Suggested OpenItem fields:
   - id.
   - source.
```

```
   - severity.
   - category.
   - title.
   - message.
   - audit_id.
   - machine.
   - field.
   - target_id.
   - due_date.
   - status.
   - recommended_action.
   - created_at.
   - updated_at.

5. Aggregate open items from available sources.
   Sources should include:
   - open notes.
   - critical/important notes.
   - active tag assignments.
   - due follow-ups.
   - validation findings if available.
   - missing evidence warnings if available.
   - compatibility concerns if available.
   - documentation gaps if available.
   - action items sheet if available.

   If some sources do not exist yet, design the aggregator to gracefully skip
them.

6. Add Open Items page.
   Suggested path:
   app/pages/open_items.py

   Features:
   - table of open items.
   - filters by source, severity, category, status, tag, audit ID, machine, due
date.
   - search box.
   - summary cards.
   - actions:
     - Open Target.
     - Mark Resolved.
     - Dismiss With Reason.
     - Add Note.
     - Create Follow-Up.
     - Export Open Items Report.

7. Add open item statuses.
```

```
   Suggested statuses:
   - Open.
   - In Progress.
   - Waiting on Info.
   - Blocked.
   - Resolved.
   - Dismissed.

8. Wire events.
   Emit or respond to events:
   - AnnotationChanged.
   - TagChanged.
   - OpenItemsChanged.
   - AuditSaved.
   - WorkbookValidated.

   Home and Audit Coach should update open item counts if easy and safe.

9. Add tests.
   Tests should cover:
   - suggestion fingerprint stability.
   - applying selected suggestions.
   - ignored suggestions hidden until data changes.
   - OpenItem aggregation from notes/tags.
   - OpenItem aggregation from validation findings if available.
   - marking item resolved/dismissed.
   - target navigation data present.

10. Update docs.
    Suggested path:
    docs/open_items_and_annotation_workflow.md

Acceptance criteria:
- Suggestions can be applied without manually opening each field tag dialog.
- Ignored suggestions do not nag forever without reason.
- Open Items page displays unresolved work from multiple sources.
- Open items can be resolved or dismissed with traceability.
- Target navigation works when target data exists.
- Existing Notes and Tags pages still work.
- Tests pass or failures are documented.
- No private data added.

Run tests:
- Run annotation backend tests.
- Run notes/tags UI tests if available.
- Run open items tests.
- Run repo safety audit if practical.
```

```
Final response requirements:
- List files changed.
- Explain suggestion apply/ignore workflow.
- Explain Open Items aggregation.
- Explain new page behavior.
- List tests run and results.
- List risks/follow-ups.
```

Phase 5 Prompt — Workbook Validation and Repair System

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 5 — Workbook Validation and Repair System from the EOAT Command
Center roadmap.

Included features:
- #9 Repairable validation findings.
- #20 Field history and audit revision history, expanded.
- #22 Local SQLite audit mirror, foundation decision/initial implementation.
- #30 Physical vs compatible row consistency checks.
- #32 Workbook lock detector.
- #33 Validation severity levels.
- #34 One-click safe fixes.

Goal:
Upgrade workbook validation from text warnings into a structured, repairable
health system.

Critical constraints:
- Preserve existing markdown validation reports.
- Do not remove existing validation behavior unless replacing it with equivalent
or better behavior.
- Safe fixes must preview changes and create backups.
- Do not guess unknown engineering values.
- Do not commit real workbooks or generated validation outputs.
- Do not implement full Robot Info entity system.

Before editing code:
1. Inspect core/validation.py.
2. Inspect workbook schema modules.
```

```
3. Inspect audit field rules.
4. Inspect audit compatibility modules.
5. Inspect audit history from Phase 2 if present.
6. Inspect safe_files/backup helpers.
7. Inspect workbook IO helpers.
8. Inspect Workbook Health page.
9. Inspect validation tests.
10. Summarize current validation categories and warning strings.

Required work:

1. Create structured ValidationFinding model.
   Suggested module:
   core/validation_findings.py

   Suggested fields:
   - finding_id.
   - severity.
   - category.
   - sheet_name.
   - row_number.
   - column_name.
   - audit_id.
   - machine_number.
   - message.
   - current_value.
   - expected_behavior.
   - recommended_action.
   - fix_available.
   - fix_id.
   - source_validator.

2. Add validation severity levels.
   Suggested levels:
   - BLOCKER.
   - ERROR.
   - WARNING.
   - INFO.
   - AUTO_FIXABLE.

   Example classifications:
   - Missing master workbook: BLOCKER.
   - Duplicate Audit ID: ERROR.
   - Applicable major field is N/A: WARNING.
   - Non-applicable field correctly stored as N/A: INFO.
   - Non-applicable hidden field has stale meaningful value: AUTO_FIXABLE.

3. Update validation to produce structured findings.
```

```
   Preserve existing ToolResult warnings and markdown summary, but add
structured findings to metrics or a new result field if existing ToolResult
supports it safely.

   If ToolResult cannot hold structured data cleanly, add a companion writer
that outputs JSON next to the markdown report.

4. Add JSON validation output.
   Output path example:
   00_Project_Admin/Validation_Reports/Foundation_Validation_YYYY-MM-
DD_HHMM.json

   Include:
   - generated_at.
   - project_root.
   - summary counts.
   - findings list.

5. Update Workbook Health page.
   Add findings table with filters:
   - severity.
   - category.
   - sheet.
   - audit ID.
   - machine.
   - field.
   - fix available.

   Actions:
   - Open Audit.
   - Jump to Field if possible.
   - Create Annotation.
   - Create Follow-Up.
   - Preview Fix.
   - Apply Safe Fix.
   - Export Report.

6. Add physical vs compatible row consistency checks.
   Suggested module:
   core/compatibility_health.py

   Check for:
   - compatible row missing source audit.
   - source audit with stale compatible rows.
   - missing compatible rows where expected.
   - extra compatible rows.
   - incompatible inherited values.
   - manually edited system-managed compatibility fields.
```

```
   Produce structured validation findings.

7. Add workbook lock detector.
   Suggested module:
   core/workbook_locks.py

   Behavior:
   - Before write operations, detect likely locked workbook.
   - Return clear status object.
   - Do not silently fail.
   - UI should offer retry/save draft/open folder where appropriate.

8. Add safe auto-fix framework.
   Suggested module:
   core/workbook_repairs.py

   Safe fixes may include:
   - clear stale hidden non-applicable fields to N/A.
   - repair known legacy headers.
   - refresh generated views.
   - reapply formatting.
   - rebuild dropdown data validation.

   Each fix must:
   - preview changes.
   - require user confirmation.
   - create backup.
   - apply fix.
   - log activity.
   - record audit history when audit rows are changed.
   - rerun validation.

   Do not implement unsafe fixes that guess engineering values.

9. Expand audit history.
   Record:
   - validation auto-fixes.
   - compatibility regenerations.
   - workbook repair actions.
   - user-initiated saves.

10. Decide on local SQLite audit mirror.
    If implementing now, start small.

    Minimum useful mirror:
    - audit IDs and key fields.
    - compatibility source relationships.
```

```
    - validation findings.
    - audit history.

    If not implementing, create a decision document explaining why and what
would be needed later.

11. Add tests.
    Tests should cover:
    - ValidationFinding creation.
    - severity classification.
    - JSON export.
    - markdown report still works.
    - stale hidden value auto-fix preview.
    - safe fix creates backup.
    - duplicate audit ID finding.
    - compatible row missing source finding.
    - workbook lock detector behavior with simulated locked/unwritable file if
possible.
    - audit history records repair events.

12. Update docs.
    Suggested path:
    docs/workbook_validation_and_repair_system.md

Acceptance criteria:
- Validation findings are structured and filterable.
- Existing validation markdown output still exists.
- JSON validation output exists.
- Workbook Health page can display findings.
- Safe fixes preview before applying.
- Safe fixes create backups.
- Workbook lock failures are clear.
- Physical vs compatible consistency issues are detectable.
- Repair actions are traceable.
- Tests pass or failures are documented.
- No private data added.

Run tests:
- Run validation tests.
- Run workbook repair tests.
- Run compatibility tests.
- Run audit history tests.
- Run repo safety audit if practical.

Final response requirements:
- List files changed.
- Explain ValidationFinding model.
- Explain severity levels.
```

```
- Explain Workbook Health UI changes.
- Explain safe auto-fix system.
- Explain compatibility health checks.
- Explain SQLite mirror decision/status.
- List tests run and results.
- List risks/follow-ups.
```

Phase 6 Prompt — Dashboard Performance, Cache, and Telemetry

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 6 — Dashboard Performance, Cache, and Telemetry from the EOAT
Command Center roadmap.

Included features:
- #13 Cache staleness improvements.
- #14 Structured performance logging.
- #22 Local SQLite audit mirror, performance use if implemented.

Goal:
Make the app faster, more transparent, and easier to diagnose when something is
slow.

Critical constraints:
- Preserve quick refresh vs deep refresh behavior.
- Quick refresh must not perform expensive workbook scans.
- Deep refresh may perform expensive recalculation, preferably in background.
- Cache must never pretend stale data is fresh.
- Do not commit generated cache or performance logs.
- Do not commit real project data.
- Do not implement full Robot Info entity system.

Before editing code:
1. Inspect core/dashboard_cache.py.
2. Inspect app/pages/home.py refresh behavior.
3. Inspect core/performance.py.
4. Inspect scheduled report status and activity log usage.
5. Inspect validation output paths if Phase 5 exists.
6. Inspect annotation database path.
7. Inspect small Robot_Info.xlsx path.
```

```
8. Inspect tests for dashboard cache/performance.
9. Summarize current cache source metadata and refresh behavior.

Required work:

1. Expand dashboard cache source tracking.
   Cache metadata should include sources that affect dashboard status:
   - master tracker workbook.
   - activity log.
   - scheduled reports log.
   - daily reports folder.
   - weekly reports folder.
   - task/schedule files.
   - annotation database.
   - small Robot_Info.xlsx workbook.
   - validation findings JSON if available.
   - photo index files if available.
   - documentation gap outputs.
   - open items outputs if available.

   Missing optional files should not break cache.

2. Add stale reason explanation.
   Instead of only returning stale true/false, return reasons.

   Example:
   Dashboard cache stale because:
   - annotations.sqlite changed.
   - EOAT_Master_Tracker.xlsx changed.
   - scheduled_tools.log changed.

3. Preserve quick refresh vs deep refresh.
   Quick Refresh:
   - Reads cache.
   - Checks lightweight metadata.
   - Does not open all workbooks.

   Deep Refresh:
   - Recomputes workbook-backed metrics.
   - Updates cache.
   - Logs performance.

4. Convert or supplement performance logging with structured JSONL.
   Suggested path:
   00_Project_Admin/logs/performance.jsonl

   Keep text log if existing UI/docs depend on it, but add structured output.
```

```
   JSONL fields:
   - timestamp.
   - operation.
   - duration_seconds.
   - success.
   - source.
   - page/tool.
   - project_root_mode.
   - details.
   - warning_count.
   - error_count.

5. Add Performance page.
   Suggested page key:
   performance

   Display:
   - slowest recent operations.
   - app startup duration.
   - dashboard quick refresh durations.
   - dashboard deep refresh durations.
   - audit save durations.
   - validation durations.
   - report generation durations.
   - cache hit/miss counts if available.
   - recent warnings/errors.

6. Add performance helpers.
   Suggested module:
   core/performance_log.py or extend core/performance.py

   Functions:
   - log_performance_event(...)
   - read_recent_performance_events(...)
   - summarize_performance(...)

7. Add tests.
   Tests should cover:
   - source metadata includes new expected sources when present.
   - stale reasons identify changed file metadata.
   - quick refresh uses cache and does not call deep collectors, if mockable.
   - JSONL performance log writes valid JSON.
   - performance summary identifies slowest operations.

8. Update docs.
   Suggested path:
   docs/performance_cache_and_telemetry.md
```

```
Acceptance criteria:
- App can show cached dashboard data quickly.
- User can tell why cache is stale.
- Deep Refresh remains intentional.
- Performance events are structured.
- Performance page shows useful diagnostics.
- Generated cache/logs remain outside repo.
- Tests pass or failures are documented.
- No private data added.

Run tests:
- Run dashboard cache tests.
- Run performance tests.
- Run Home page smoke tests if available.
- Run repo safety audit if practical.

Final response requirements:
- List files changed.
- Explain cache metadata changes.
- Explain stale reason behavior.
- Explain performance logging changes.
- Explain Performance page.
- List tests run and results.
- List risks/follow-ups.
```

Phase 7 Prompt — Scheduled Reports and Report Reliability

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 7 — Scheduled Reports and Report Reliability from the EOAT
Command Center roadmap.

Included features:
- #15 Scheduled report calendar preview.
- #16 Catch-up summaries.
- #17 Smarter scheduled report content.
- #18 Task Scheduler preflight.

Goal:
Make automated daily and weekly reports visible, testable, recoverable, and
```

```
genuinely useful.

Critical constraints:
- Preserve existing scheduled report scripts.
- Preserve manual daily/weekly summary commands.
- Do not overwrite existing reports.
- Do not require the dashboard to be open for scheduled tasks.
- Do not commit generated reports/logs.
- Do not commit real project data.
- Do not implement full Robot Info entity system.

Before editing code:
1. Inspect core/scheduled_reports.py.
2. Inspect app/pages/scheduled_reports.py.
3. Inspect scripts/install_summary_schedules.ps1.
4. Inspect scripts/check_summary_schedules.ps1.
5. Inspect scripts/run_daily_summary.ps1.
6. Inspect scripts/run_weekly_summary.ps1.
7. Inspect daily_status_summary.py.
8. Inspect core/weekly_summary.py.
9. Inspect activity logging.
10. Inspect schedule/task progress modules.
11. Summarize current scheduled report behavior and duplicate prevention.

Required work:

1. Upgrade Scheduled Reports page status.
   Show:
   - daily task installed status.
   - weekly task installed status.
   - last run status.
   - last generated daily report.
   - last generated weekly report.
   - next expected run.
   - missed report dates.
   - scheduled log path.
   - output folder paths.

2. Add scheduled report calendar preview.
   Suggested function:
   core/scheduled_reports.py:
   preview_summary_schedule(project_root, start_date=None, days=14,
timezone_name=...)

   Each preview row should include:
   - date.
   - weekday.
   - expected automation type.
```

```
   - scheduled time.
   - status: not scheduled, already exists, due, future, missed, skipped.
   - existing report path if present.
   - decision reason.

   UI should show 14 or 30 days.

3. Add catch-up summary workflow.
   If reports were missed, allow user to select missed dates and generate
reports.

   Requirements:
   - Use correct target report date.
   - Resolve correct project week/day.
   - Do not overwrite existing report.
   - Clearly mark dry-run/catch-up behavior if appropriate.
   - Log activity.

   Actions:
   - Generate selected missed daily reports.
   - Generate selected missed weekly reports.
   - Mark selected intentionally skipped if implementing skipped records.
   - Open generated reports.

4. Add Task Scheduler preflight diagnostics.
   Check:
   - Windows platform.
   - powershell.exe available.
   - Task Scheduler commands accessible.
   - daily task exists.
   - weekly task exists.
   - scripts exist.
   - Python executable exists.
   - project root exists.
   - output folders writable.
   - scheduled log readable.

   Show status in UI with clear pass/warning/error text.

5. Add manual dry-run buttons.
   Buttons:
   - Run Daily Dry Run.
   - Run Weekly Dry Run.
   - Generate Daily Now.
   - Generate Weekly Now.
   - Install/Repair Tasks.
   - Uninstall Tasks.
   - Open Logs.
```

```
   - Copy Diagnostics.

6. Improve scheduled report content inputs.
   Create report context builder.
   Suggested module:
   core/report_context.py

   Daily report context should use:
   - activity log.
   - completed tasks.
   - audit saves.
   - annotations created.
   - validation findings.
   - open items.
   - reports generated.
   - schedule progress.
   - blockers.
   - changed files.

   Weekly report context should use:
   - daily reports.
   - weekly activity.
   - audit progress delta.
   - open item changes.
   - validation changes.
   - pilot/FMEA/KPI changes if available.

   Reports should avoid generic filler when real context exists.

7. Preserve script behavior.
   Existing PowerShell scripts should still work.
   If script parameters are added, keep backward compatibility.

8. Add tests.
   Tests should cover:
   - calendar preview for Monday-Thursday daily.
   - calendar preview for Friday weekly.
   - duplicate report detection.
   - missed daily detection.
   - catch-up report target date/week/day.
   - preflight behavior with mocked missing PowerShell/task status.
   - report context uses activity log data.

9. Update docs.
   Suggested path:
   docs/scheduled_reports_reliability.md

Acceptance criteria:
```

```
- User can see whether scheduled tasks are installed.
- User can see upcoming scheduled report runs.
- Missed reports are visible.
- Missed reports can be generated manually without overwriting existing files.
- Dry runs are available.
- Reports include real project activity when available.
- Existing scheduled scripts still work.
- Tests pass or failures are documented.
- No private data added.

Run tests:
- Run scheduled report tests.
- Run summary generator tests.
- Run dashboard page tests if available.
- Run repo safety audit if practical.

Final response requirements:
- List files changed.
- Explain calendar preview.
- Explain catch-up workflow.
- Explain preflight diagnostics.
- Explain report context improvements.
- List tests run and results.
- List risks/follow-ups.
```

Phase 8 Prompt — Search, Navigation, and Productivity Layer

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 8 — Search, Navigation, and Productivity Layer from the EOAT
Command Center roadmap.

Included features:
- #11 Global command palette.
- #12 Global search.
- #23 SQLite FTS search.
- #31 App-native Press View, navigation side.

Goal:
Make the app easier to navigate as pages, audits, reports, notes, tags, and
```

```
findings grow.

Critical constraints:
- Do not break current navigation.
- Search must tolerate missing optional data sources.
- Destructive commands must require confirmation.
- Do not commit search indexes/cache databases generated from real data.
- Do not commit real project data.
- Do not implement full Robot Info entity system.

Before editing code:
1. Inspect page registry from Phase 1.
2. Inspect annotations search methods.
3. Inspect audit row loading/listing methods.
4. Inspect report listing methods.
5. Inspect validation findings if Phase 5 exists.
6. Inspect Press/Audit by Press modules.
7. Inspect existing widgets/dialogs style.
8. Summarize current navigation and searchable data sources.

Required work:

1. Add command registry.
   Suggested module:
   core/commands.py or app/command_registry.py

   Each command should include:
   - command_id.
   - display_name.
   - aliases.
   - category.
   - handler.
   - safety_level.
   - requires_confirmation.
   - enabled/disabled state.

   Command categories:
   - Navigation.
   - Audit.
   - Reports.
   - Validation.
   - Settings.
   - Scheduled Reports.
   - Open Items.

2. Add global command palette.
   Suggested widget:
   app/widgets/command_palette.py
```

```
   Shortcut:
   Ctrl+K

   Commands should include:
   - open Home.
   - open EOAT Audit.
   - open Open Items.
   - open Workbook Health.
   - open Scheduled Reports.
   - open Settings.
   - run workbook validation.
   - deep refresh dashboard.
   - generate daily summary.
   - generate weekly summary.
   - open latest report.
   - open project folder.

   Destructive/file-modifying commands require confirmation.

3. Add global search service.
   Suggested module:
   core/search.py

   Search result fields:
   - result_id.
   - result_type.
   - title.
   - subtitle.
   - detail.
   - audit_id.
   - machine.
   - field.
   - target_id.
   - path.
   - action.

   Search across:
   - audits.
   - machine/press numbers.
   - notes.
   - tags.
   - open items.
   - validation findings.
   - reports.
   - photo index records if available.

4. Add search filters.
```

```
   Filters:
   - type.
   - audit ID.
   - machine/press.
   - tag.
   - status.
   - severity.
   - date.
   - due date.

   Keep initial implementation simple if needed.

5. Add SQLite FTS if appropriate.
   If the annotation database or SQLite mirror exists, add FTS for notes/tags.
   If not appropriate yet, add a documented fallback using existing LIKE search.

   Do not block Phase 8 on full FTS if it would cause too much churn.

6. Add app-native Press View.
   Suggested page:
   app/pages/press_view.py

   For each press/machine show:
   - physical audits.
   - compatible entries.
   - tools.
   - open items.
   - validation warning counts.
   - photo coverage if available.
   - pilot candidacy if available.
   - last updated date.

   Actions:
   - open audit.
   - open machine group.
   - filter by status.
   - export press summary.

7. Wire search and command results to navigation.
   Search results should open relevant pages/targets:
   - audit result opens audit.
   - field result opens audit and focuses field if supported.
   - note/tag result opens Notes/Tags target.
   - report result opens file.
   - press result opens Press View.

8. Add tests.
   Tests should cover:
```

```
   - command registry contains expected commands.
   - command palette filtering works.
   - destructive command requires confirmation flag.
   - search returns audit results.
   - search returns notes/tags results.
   - search tolerates missing optional data sources.
   - Press View grouping separates physical and compatible entries.

9. Update docs.
   Suggested path:
   docs/search_command_palette_and_press_view.md

Acceptance criteria:
- Ctrl+K opens command palette.
- User can quickly navigate to important pages/tools.
- Search returns audits, notes, tags, findings, reports, and machines when data
exists.
- Press View summarizes audits by machine/press.
- Destructive commands require confirmation.
- Tests pass or failures are documented.
- No private data added.

Run tests:
- Run command/search tests.
- Run Press View tests.
- Run UI smoke tests if available.
- Run repo safety audit if practical.

Final response requirements:
- List files changed.
- Explain command palette.
- Explain global search.
- Explain Press View.
- Explain FTS decision/status.
- List tests run and results.
- List risks/follow-ups.
```

Phase 9 Prompt — Evidence, Photos, PM, BOM, and Documentation Coverage

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 9 — Evidence, Photos, PM, BOM, and Documentation Coverage from
the EOAT Command Center roadmap.

Included features:
- #8 Gripper presets mini parts database.
- #24 Photo evidence coverage.
- #25 Phone-friendly photo intake.

Goal:
Improve audit evidence quality and documentation completeness.

Critical constraints:
- Do not commit real photos.
- Do not commit real photo indexes from private projects.
- Use synthetic/demo photo placeholders only in demo data.
- Preserve existing photo intake behavior.
- Gripper presets should be configurable/reference data, not hardcoded only in
UI.
- Do not implement full Robot Info entity system.
- Do not commit real company data.

Before editing code:
1. Inspect photo page and photo indexing modules.
2. Inspect audit fields related to photos/documentation.
3. Inspect gripper fields module.
4. Inspect PM checklist and BOM/spare parts modules.
5. Inspect documentation gap modules.
6. Inspect validation modules if structured findings exist.
7. Summarize current photo/documentation/gripper behavior.

Required work:

1. Add photo evidence category model.
   Suggested module:
   core/photo_evidence.py

   Suggested categories:
   - overall_eoat.
```

```
   - robot_connection.
   - eoat_pneumatic_circuits.
   - sensors.
   - quick_disconnects.
   - tubing_routing.
   - grippers.
   - vacuum_cups.
   - mounting_hardware.
   - cable_management.
   - wear_damage.
   - process_binder_reference.

   Each category should define:
   - key.
   - label.
   - applies_when.
   - required_when.
   - recommended_when.
   - example filename prefix.

2. Add evidence coverage per audit.
   Create status model:
   - audit_id.
   - category.
   - applies.
   - required.
   - present.
   - photo_count.
   - status: complete, partial, missing, not applicable, follow-up needed.
   - warning.

3. Add evidence UI.
   Add to Audit page or Photos page:
   - evidence coverage table.
   - required/missing categories.
   - open folder button.
   - create intake folder button.
   - associate existing photos if supported.

4. Add evidence validation warnings.
   Validation should warn when:
   - audit is Complete but required evidence is missing.
   - pilot candidate lacks before photos.
   - issue has no supporting evidence.
   - documentation marked complete but no document/photo link exists.
   - Sensors Present? is Yes but no sensor photo exists.
   - Quick Disconnects Present? is Yes but no quick disconnect photo exists.
```

```
   Use structured findings if Phase 5 exists.

5. Add phone-friendly photo intake workflow.
   Minimum implementation:
   - create audit-specific intake folder.
   - show folder path.
   - copy folder path.
   - generate photo checklist markdown.
   - include expected categories.

   Suggested output:
   01_EOAT_Audit/Photos/Incoming/AUD-YYYY.../

   Optional later/future note:
   - local web upload page.
   - QR code.
   - same-network upload.

   Do not implement network upload unless it can be done safely and locally.

6. Move gripper presets to configurable reference data.
   Suggested file:
   data_templates/gripper_presets.example.json
   or config/reference/gripper_presets.json if config pattern exists.

   Each preset:
   - friendly_name.
   - part_number.
   - manufacturer.
   - default_type.
   - notes.
   - active.

   Preserve current mappings:
   - Large Double Gripper -> MHZL2-16D.
   - Small Double Gripper -> MHZL2-10S.

   UI should still show friendly names and save workbook part/model number.

7. Add PM/BOM coverage hooks.
   Do not necessarily rebuild PM/BOM pages.
   Add reusable functions so PM/BOM workflows can ask:
   - Is spare parts info missing?
   - Is BOM available?
   - Is gripper preset known?
   - Are standard parts opportunities present?
   - Is documentation/photo evidence missing?
```

```
8. Add tests.
   Tests should cover:
   - evidence category applicability for Vacuum EOAT.
   - evidence category applicability for Mechanical / Gripper EOAT.
   - evidence category applicability for Hybrid EOAT.
   - missing required evidence finding.
   - intake folder/checklist generation.
   - gripper preset loading.
   - friendly name to part number mapping still works.
   - no real photo files required.

9. Update docs.
   Suggested path:
   docs/photo_evidence_and_gripper_presets.md

Acceptance criteria:
- Each audit can show photo evidence completeness.
- Missing evidence is visible and actionable.
- Phone/photo intake is less manual.
- Gripper presets are not only hardcoded in Python.
- Existing gripper model workbook mapping still works.
- PM/BOM workflows can reuse coverage hooks.
- Tests pass or failures are documented.
- No real photos or private data added.

Run tests:
- Run photo/evidence tests.
- Run gripper preset tests.
- Run validation tests if evidence warnings integrate there.
- Run repo safety audit if practical.

Final response requirements:
- List files changed.
- Explain evidence categories.
- Explain photo intake workflow.
- Explain gripper preset config.
- Explain PM/BOM hooks.
- List tests run and results.
- List risks/follow-ups.
```

Phase 10 Prompt — Standards, FMEA, Pilot, and Engineering Analysis

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 10 — Standards, FMEA, Pilot, and Engineering Analysis from the
EOAT Command Center roadmap.

Included features:
- #26 Pilot candidate evidence packet, analysis side.
- #27 FMEA generation from tags and issues.
- #28 Standards compliance score.
- #31 App-native Press View, analysis side.

Goal:
Turn collected audit data into engineering analysis and improvement
recommendations.

Critical constraints:
- Do not fabricate engineering conclusions.
- FMEA suggestions must be reviewable and editable by the user.
- Standards compliance must treat Unknown, N/A, and verified values differently.
- Do not silently convert suggestions into final engineering decisions.
- Do not commit real analysis outputs from private project data.
- Do not implement full Robot Info entity system.

Before editing code:
1. Inspect FMEA modules/page.
2. Inspect pilot candidate modules/page.
3. Inspect standards/docs modules/page.
4. Inspect issue analysis modules/page.
5. Inspect Open Items if Phase 4 exists.
6. Inspect photo evidence if Phase 9 exists.
7. Inspect validation findings if Phase 5 exists.
8. Inspect Press View if Phase 8 exists.
9. Summarize current analysis and report capabilities.

Required work:

1. Add standards compliance model.
   Suggested module:
   core/standards_compliance.py
```

```
   Categories:
   - EOAT classification complete.
   - tooling details complete.
   - pneumatic routing condition.
   - sensor standard/documentation.
   - quick disconnect standard.
   - cable management.
   - mechanical mounting.
   - safety concerns.
   - documentation completeness.
   - PM readiness.
   - BOM/spare parts readiness.
   - photo evidence readiness.

   Each category should output:
   - status: compliant, warning, fail, not applicable, unknown.
   - score.
   - reason.
   - recommended action.
   - related fields.

2. Add compliance score per audit.
   Show:
   - overall score.
   - category scores.
   - failed standards.
   - warnings.
   - unknown/follow-up items.
   - recommended actions.

3. Add compliance summary by press/cell.
   If Press View exists, add:
   - average compliance score.
   - worst category.
   - open standards issues.
   - pilot candidate relevance.

4. Generate FMEA suggestions from real data.
   Suggested module:
   core/fmea_suggestions.py

   Inputs:
   - known issues.
   - drop/mis-pick history.
   - maintenance frequency.
   - validation findings.
   - tags.
   - notes.
```

```
   - open items.
   - photo evidence gaps.
   - standards compliance failures.
   - sensor/pneumatic/mechanical fields.

   Suggested failure modes:
   - vacuum loss.
   - misalignment.
   - tubing failure.
   - sensor failure.
   - mechanical wear.
   - quick disconnect mismatch.
   - documentation failure.
   - maintenance access issue.

   The user must approve/edit severity, occurrence/frequency, and detection.

5. Add FMEA suggestion UI.
   Show suggestion table:
   - Accept.
   - Failure mode.
   - Evidence.
   - Suggested severity.
   - Suggested occurrence/frequency.
   - Suggested detection.
   - Suggested mitigation.
   - Source fields/tags.

   Buttons:
   - Accept Selected.
   - Edit Before Accepting.
   - Reject Selected.
   - Export Draft.

6. Add pilot candidate evidence packet generator.
   Suggested module:
   core/pilot_evidence_packets.py

   Packet should include:
   - machine/press.
   - audit ID.
   - EOAT type.
   - known issues.
   - failure modes.
   - standards gaps.
   - downtime/scrap/cycle-time concerns if available.
   - photo/evidence coverage.
   - open items.
```

```
   - expected improvement area.
   - implementation difficulty.
   - risks.
   - recommended next action.

   Output format:
   - Markdown first.
   - Optional Excel/DOCX later only if existing patterns make it easy.

7. Add analysis exports.
   Exports:
   - compliance summary.
   - FMEA draft.
   - top failure modes.
   - pilot candidate packet.
   - press/cell risk summary.

8. Add tests.
   Tests should cover:
   - compliance scoring for Vacuum EOAT.
   - compliance scoring for Mechanical / Gripper EOAT.
   - compliance scoring for Hybrid EOAT.
   - Unknown vs N/A scoring.
   - FMEA suggestion generated from known issue/tag.
   - FMEA suggestions require review/acceptance.
   - pilot evidence packet includes expected sections.
   - press compliance rollup.

9. Update docs.
   Suggested path:
   docs/standards_fmea_pilot_analysis.md

Acceptance criteria:
- Standards scoring uses real audit data.
- Unknown and N/A are treated honestly.
- FMEA suggestions are reviewable, not auto-final.
- Pilot evidence packets are evidence-backed.
- Analysis exports are created safely without overwriting prior outputs.
- Tests pass or failures are documented.
- No private data added.

Run tests:
- Run standards compliance tests.
- Run FMEA tests.
- Run pilot candidate tests.
- Run Press View tests if affected.
- Run repo safety audit if practical.
```

```
Final response requirements:
- List files changed.
- Explain standards compliance scoring.
- Explain FMEA suggestion workflow.
- Explain pilot evidence packets.
- Explain analysis exports.
- List tests run and results.
- List risks/follow-ups.
```

Phase 11 Prompt — Final Handoff and Leadership-Ready Outputs

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 11 — Final Handoff and Leadership-Ready Outputs from the EOAT
Command Center roadmap.

Included features:
- #26 Pilot candidate evidence packet, final output side.

Goal:
Prepare the app to generate clean final deliverables for the end of the EOAT
standardization project.

Critical constraints:
- Do not overwrite existing final handoff packages.
- Do not commit generated real final reports.
- Do not include private company data in repo docs or demo outputs.
- Outputs generated from real project data must stay inside the private project
root.
- Do not fabricate KPI impact or pilot results.
- Do not implement full Robot Info entity system.

Before editing code:
1. Inspect Final Handoff page/modules.
2. Inspect report-generation modules.
3. Inspect FMEA outputs.
4. Inspect KPI dashboard/export modules.
5. Inspect PM checklist generation.
6. Inspect standards docs page/modules.
7. Inspect pilot evidence packet outputs from Phase 10 if present.
```

```
8. Inspect Open Items page/data if Phase 4 exists.
9. Inspect documentation gap modules.
10. Summarize current final handoff capabilities.

Required work:

1. Upgrade Final Handoff page.
   It should show final deliverable readiness:
   - EOAT Database.
   - Standards Guidelines.
   - PM Checklist Package.
   - FMEA Output.
   - KPI Dashboard/Export.
   - Pilot Results or Pilot Candidate Packets.
   - Training Materials.
   - Documentation Gap Summary.
   - Open Items Carryover.
   - Executive Summary.
   - Technical Appendix.

2. Add final deliverable checklist model.
   Suggested module:
   core/final_handoff_readiness.py

   Each deliverable should include:
   - key.
   - label.
   - status: ready, draft, missing, needs review, not applicable.
   - evidence/source path.
   - warnings.
   - recommended action.

3. Add leadership summary export.
   Suggested output:
   Executive_Summary.md

   Include:
   - project objective.
   - work completed.
   - major findings.
   - pilot recommendation/results.
   - KPI impact if available.
   - remaining risks.
   - next steps.

   Rule:
   - If KPI impact is unavailable, say unavailable. Do not fabricate numbers.
```

```
4. Add technical appendix export.
   Suggested output:
   Technical_Appendix.md

   Include:
   - audit coverage.
   - validation findings summary.
   - standards gaps.
   - FMEA details.
   - PM/BOM findings.
   - photo/evidence references.
   - open items.
   - compatibility health summary.

5. Add final handoff package builder.
   Output folder:
   06_Final_Handoff/Final_Handoff_Package_YYYYMMDD_HHMM/

   Suggested structure:
   - Executive_Summary.md
   - Technical_Appendix.md
   - Open_Items_Carryover.md
   - Deliverable_Readiness.md
   - FMEA/
   - KPI/
   - PM_Checklists/
   - Pilot_Candidates/
   - Standards/
   - Validation/

   Do not overwrite existing package folders.

6. Add open items carryover export.
   Include unresolved:
   - notes.
   - tags.
   - validation findings.
   - missing evidence.
   - documentation gaps.
   - follow-ups.

7. Add leadership-friendly wording.
   Keep the executive summary concise and readable.
   Keep the technical appendix detailed.
   Do not use snark or joke tone in generated business deliverables.

8. Add tests.
   Tests should cover:
```

```
   - readiness checklist status for missing/ready outputs.
   - package folder creation without overwrite.
   - executive summary includes required headings.
   - technical appendix includes required headings.
   - open items carryover export.
   - unavailable KPI/pilot data is stated honestly.

9. Update docs.
   Suggested path:
   docs/final_handoff_outputs.md

Acceptance criteria:
- Final Handoff page shows readiness clearly.
- Final package builder creates dated folder.
- Existing packages are not overwritten.
- Executive and technical outputs are separated.
- Open items are not hidden.
- Missing/unavailable data is stated honestly.
- Tests pass or failures are documented.
- No private data added to repo.

Run tests:
- Run final handoff tests.
- Run report generation tests.
- Run open items tests if affected.
- Run repo safety audit if practical.

Final response requirements:
- List files changed.
- Explain final handoff readiness model.
- Explain package structure.
- Explain executive/technical exports.
- List tests run and results.
- List risks/follow-ups.
```

Phase 12 Prompt — Repo Safety, Release Readiness, and Long-Term Maintenance

Copy/Paste Codex Prompt

```
You are working in the EOAT Command Center repository.

Implement Phase 12 — Repo Safety, Release Readiness, and Long-Term Maintenance
```

```
from the EOAT Command Center roadmap.

Included features:
- #21 Backup retention cleanup.
- #39 Release readiness page.

Goal:
Make the project safer to maintain, safer to push to GitHub, and easier to hand
off or continue later.

Critical constraints:
- Do not delete backups without preview and confirmation.
- Do not commit real data.
- Repo safety scanner should be conservative.
- Do not block safe demo/template files.
- Do not implement full Robot Info entity system.

Before editing code:
1. Inspect scripts/repo_safety_audit.py.
2. Inspect scripts/pre_commit_check.ps1 if present.
3. Inspect .gitignore.
4. Inspect README/USAGE safety sections.
5. Inspect backup_file and safe_files modules.
6. Inspect generated backup folder patterns.
7. Inspect any settings/safety pages.
8. Inspect tests for repo safety.
9. Summarize current repo safety and backup behavior.

Required work:

1. Add Backup Manager.
   Suggested module:
   core/backup_manager.py

   It should discover workbook backup folders and report:
   - backup count.
   - total backup size.
   - oldest backup.
   - newest backup.
   - backups by source workbook.
   - cleanup candidates.

2. Add backup retention policy.
   Default policy:
   - keep all backups from last 7 days.
   - keep last 25 backups per workbook.
   - keep milestone backups if identifiable.
   - never delete backups if latest validation has blockers.
```

```
   - require confirmation before cleanup.

   Cleanup should have:
   - preview mode.
   - apply mode.
   - dry-run support.
   - activity log entry.

3. Add Backup Manager UI.
   Could be a dedicated page or Settings section.

   Show:
   - backup counts.
   - total size.
   - cleanup candidates.
   - retention policy.

   Buttons:
   - Preview Cleanup.
   - Clean Safe Old Backups.
   - Open Backup Folder.

4. Add Release Readiness page.
   Suggested page:
   app/pages/release_readiness.py

   Checks:
   - tests pass or last test status known.
   - repo safety audit pass.
   - app smoke test pass if available.
   - no real workbooks staged.
   - no real photos staged.
   - no local config staged.
   - no generated reports/logs/cache staged.
   - README/USAGE exists.
   - demo project exists.
   - git status visible.
   - branch status visible if available.

   Buttons:
   - Run Tests.
   - Run Repo Safety Audit.
   - Show Staged Files.
   - Open Sanitization Report.
   - Copy Commit Checklist.

5. Improve staged-file safety scanner.
   Extend existing safety audit if possible.
```

```
   Flag blockers:
   - .xlsx files outside allowed demo/template paths.
   - real project root folders.
   - generated reports.
   - local config files.
   - logs/cache files.
   - photos.
   - private network paths.
   - suspicious internal paths.

   Flag warnings:
   - large binary files.
   - public company references combined with operational context.
   - suspicious identifiers if pattern-based detection exists.

6. Add optional pre-commit hook installer.
   Provide script or UI action to install a local pre-commit hook that runs
safety checks.

   Requirements:
   - local only.
   - documented.
   - should not be required for app use.
   - should fail closed on obvious blockers.

7. Add maintenance documentation.
   Suggested path:
   docs/maintenance_and_release_readiness.md

   Include:
   - adding new pages.
   - adding new tools.
   - adding validation rules.
   - adding settings.
   - adding report generators.
   - running tests.
   - preparing safe GitHub commits.
   - backup retention behavior.

8. Add tests.
   Tests should cover:
   - backup discovery.
   - retention policy candidate selection.
   - cleanup dry run does not delete.
   - scanner flags unsafe workbook paths.
   - scanner allows demo/template files.
   - scanner flags local config.
```

```
   - release readiness summary handles missing git gracefully.

Acceptance criteria:
- User can see backup counts and cleanup candidates.
- Cleanup preview works before deletion.
- Release readiness is visible in app.
- Unsafe repo content is flagged before commit/push.
- Demo/template files are not falsely blocked.
- Maintenance documentation exists.
- Tests pass or failures are documented.
- No private data added.

Run tests:
- Run backup manager tests.
- Run repo safety tests.
- Run release readiness tests.
- Run full repo safety audit.
- Run broader pytest if practical.

Final response requirements:
- List files changed.
- Explain backup manager and retention policy.
- Explain release readiness page.
- Explain safety scanner improvements.
- Explain pre-commit hook option.
- List tests run and results.
- List risks/follow-ups.
```

Practical Execution Strategy

Recommended Run Order

Use the phase prompts in this order:

1. Phase 0 — Current-State Audit and Safety Baseline.
2. Phase 1 — App Architecture Foundation.
3. Phase 2 — Audit Workflow Stabilization.
4. Phase 3 — Audit Coach and Guided Completion.
5. Phase 4 — Annotations, Notes, Tags, and Open Items.
6. Phase 5 — Workbook Validation and Repair System.
7. Phase 6 — Dashboard Performance, Cache, and Telemetry.
8. Phase 7 — Scheduled Reports and Report Reliability.
9. Phase 8 — Search, Navigation, and Productivity Layer.
10. Phase 9 — Evidence, Photos, PM, BOM, and Documentation Coverage.

11. Phase 10 — Standards, FMEA, Pilot, and Engineering Analysis.
12. Phase 11 — Final Handoff and Leadership-Ready Outputs.
13. Phase 12 — Repo Safety, Release Readiness, and Long-Term Maintenance.

If Time Is Limited

Prioritize:

1. Phase 2 — Audit Workflow Stabilization.
2. Phase 3 — Audit Coach and Guided Completion.
3. Phase 5 — Workbook Validation and Repair System.
4. Phase 7 — Scheduled Reports and Report Reliability.
5. Phase 4 — Open Items Board.
6. Phase 6 — Performance and Cache.

Phase 1 should happen early, but it can be implemented lightly if necessary.

Recommended Codex Session Pattern

For each phase:

1. Paste the phase prompt.
2. Tell Codex to inspect first and summarize before major edits.
3. Review its plan.
4. Let it implement.
5. Require tests.
6. Review changed files.
7. Run the app manually if needed.
8. Only then move to the next phase.

Do not let Codex combine phases unless the previous phase was completed cleanly. That path leads to a Python swamp, and nobody wants to discover a new species of bug in `audit_page_final_FINAL_revised2.py`.

Final Reminder

The target app should help answer:

- What still needs to be audited?
- What information is missing?
- What evidence is missing?
- Which warnings actually matter?
- Which EOATs are risky?
- Which standards are not being met?
- Which pilot candidates are strongest?

- What reports are due?
- What is ready for final handoff?
- What is unsafe to commit?

The app should not merely store audit data. It should guide the project, protect the user’s work, and produce a clean final handoff.

In other words: fewer spreadsheet goblins, more engineering command center.